

Abaqus Python Scripting Fundamentals

20-Day Plan (30 Minutes/Day)

Daily 30-Minute Session Template (Use Every Day)

1. 0–3 min: Recall

- Open yesterday's script and run it once.
- Confirm the environment and paths are correct (CAE vs ODB mode).

2. 3–25 min: One focused change

- Implement exactly one concept (new API object, one refactor, one extractor).
- Keep the model small (single part, single step) unless the day expands scope.

3. 25–30 min: Lock it in

- Add short notes: what you learned + key object paths.
- Save a clean runnable version (and a scratch copy if needed).
- Write 3 bullets: what worked / what broke / what next.

Project Folder Layout (Build This Over Time)

- `01_build_model.py` (CAE/kernel model build)
- `02_run_jobs.py` (job submission + parameter loop)
- `03_post_odb.py` (ODB extraction)
- `utils.py` (helpers: naming, selection, IO, logging)
- `README.md` (how to run + conventions)

Recommended Training Model

Use one consistent model for all 20 sessions:

- **Cantilever plate/beam with a hole** (good for sets, robust selections, mesh control, stress extraction).
- Start with **linear static, implicit** for Weeks 1–2.

1 Week 1 (Days 1–5): Setup + First End-to-End Run

Day 1: Run Modes + Skeleton Script

- **Goal:** Run a script in CAE and with `noGUI`.
- **Focus change:** Create a minimal runnable script skeleton.
- **Tasks (3–25 min):**
 - Create a new model in `mdb`, print basic info (model names).
 - Save a CAE file to a known folder.
- **Deliverable:** `01_build_model.py` runs without errors in CAE.

Day 2: Replay → Refactor (First Contact)

- **Goal:** Learn from `abacus.rpy` without copying brittle lines.
- **Focus change:** Create a part (plate/block) via clean scripted commands.
- **Tasks (3–25 min):**
 - Build simple geometry manually once, then inspect `abacus.rpy`.
 - Copy only essential creation lines into `01_build_model.py`.
 - Refactor: replace masks with clearer operations where possible.
- **Deliverable:** Script creates the part consistently.

Day 3: Assembly + Step + BC/Load

- **Goal:** Touch the core pre-processing objects.
- **Focus change:** Add instance, one analysis step, and one BC/load.
- **Tasks (3–25 min):**
 - Create assembly instance.
 - Create one step (e.g., `StaticStep`).
 - Apply one displacement BC at the fixed end; apply one load (pressure/force).
- **Deliverable:** Model builds to a runnable state (even if not meshed).

Day 4: Mesh + Element Types

- **Goal:** Script basic meshing reliably.
- **Focus change:** Seed + element assignment + mesh generation.
- **Tasks (3–25 min):**
 - Seed the part/instance with a single global size.
 - Assign a suitable element type (e.g., `CPS4R` / `C3D8R` depending on model).
 - Generate mesh and verify element count is nonzero.
- **Deliverable:** Script generates mesh without manual steps.

Day 5: Job Creation + Submit + Wait

- **Goal:** One end-to-end run.
- **Focus change:** Create and run a job and handle failures.
- **Tasks (3–25 min):**
 - Create `mdb.Job`, `submit`, `waitForCompletion()`.
 - Save output to a stable folder; avoid overwriting.
 - If job fails, capture the error and make a minimal fix.
- **Deliverable:** One valid `.odb` produced.

2 Week 2 (Days 6–10): Robust Selections + Parameterization

Day 6: Stop Using Masks (Robust Selections)

- **Goal:** Make scripts resilient to geometry changes.
- **Focus change:** Replace mask/index-based selections.
- **Tasks (3–25 min):**
 - Create named sets/surfaces: `SET_FIXED`, `SET_LOAD`, etc.
 - Use `findAt` or bounding boxes where appropriate.
- **Deliverable:** BCs/loads reference sets, not masks.

Day 7: Parameterize Geometry

- **Goal:** Controlled inputs.
- **Focus change:** Move key dimensions into a `params` dictionary.
- **Tasks (3–25 min):**
 - Define `params = {L, W, t, holeR}` (or relevant set).
 - Regenerate geometry from `params`.
- **Deliverable:** Changing one value changes the geometry correctly.

Day 8: Parameterize Mesh + Step Controls

- **Goal:** Basic numerical controls as parameters.
- **Focus change:** Mesh seed size and step controls driven by `params`.
- **Tasks (3–25 min):**
 - Add `meshSize` to `params`.
 - Add step settings you commonly tune (increments/output frequency).
- **Deliverable:** “coarse vs fine” switch works from one variable.

Day 9: Functions + File Structure

- **Goal:** Maintainability.
- **Focus change:** Refactor into a pipeline of functions.
- **Tasks (3–25 min):**
 - Create: `build_part()`, `build_assembly()`, `apply_bcs()`, `mesh()`, `make_job()`.
 - Put small helpers into `utils.py`.
- **Deliverable:** `01_build_model.py` is readable and modular.

Day 10: Batch Jobs (2–5 Cases)

- **Goal:** Parametric study workflow.
- **Focus change:** Loop over parameter sets and run jobs safely.
- **Tasks (3–25 min):**
 - Generate unique job names (include param values).
 - Create per-case output folders; avoid overwriting ODBs.
 - Run and record status per case.
- **Deliverable:** Multiple ODBs produced from one run.

3 Week 3 (Days 11–15): ODB Fundamentals (Post-Processing)

Day 11: Open ODB + Navigate Steps/Frames

- **Goal:** Understand the ODB object hierarchy.
- **Focus change:** Open ODB, list steps, select last frame.
- **Tasks (3–25 min):**
 - Open one ODB, print steps and frame count.
 - Print available field outputs at last frame.
- **Deliverable:** `03_post_odb.py` prints a clear ODB summary.

Day 12: Field Output Extraction (e.g., von Mises Max)

- **Goal:** Extract scalar metrics from field output.
- **Focus change:** Compute max Mises (global or within a set/region).
- **Tasks (3–25 min):**
 - Get stress field; compute von Mises.
 - Restrict to an element set if possible.
 - Add guards for missing fields / empty regions.
- **Deliverable:** Prints one scalar per ODB (max Mises).

Day 13: History Output Extraction (RF/U vs Time)

- **Goal:** Work with XY/time histories.
- **Focus change:** Extract one history curve and write CSV.
- **Tasks (3–25 min):**
 - Identify a reference point or node set for RF/U.
 - Extract (time, value) pairs and save to `results.csv`.
- **Deliverable:** CSV written for one ODB.

Day 14: Batch Post (Loop Over ODBs)

- **Goal:** Automate extraction over many cases.
- **Focus change:** Loop all ODBs from Day 10 and build a summary table.
- **Tasks (3–25 min):**
 - For each ODB: extract max Mises + one history metric (peak RF, etc.).
 - Write `summary.csv` with job name and metrics.
- **Deliverable:** `summary.csv` produced for all cases.

Day 15: Basic Report Output (Optional Visual Automation)

- **Goal:** Repeatable visualization or report artifacts.
- **Focus change:** Auto-save one contour image OR export an XY plot.
- **Tasks (3–25 min):**
 - Create a viewport/display state and save a PNG, or
 - Export an XY curve to file for each ODB.
- **Deliverable:** Images or XY exports created without manual steps.

4 Week 4 (Days 16–20): Integration + Production Habits

Day 16: Command-Line Arguments

- **Goal:** Run scripts like tools.
- **Focus change:** Add argument parsing to control parameters/output.
- **Tasks (3–25 min):**
 - Parse args after `--` (e.g., `--mesh 2.0 --outdir out`).
 - Defaults if args are missing; validate numeric ranges.
- **Deliverable:** Run with args changes behavior predictably.

Day 17: Logging + Error Handling

- **Goal:** Clear failures and traceability.
- **Focus change:** Add logging to file and robust try/except around runs.
- **Tasks (3–25 min):**
 - Create `run.log`; log case parameters + job status.
 - Record failed cases and continue to next case.
- **Deliverable:** Failures produce useful logs, not silent exits.

Day 18: Clean Param Study (10 Cases)

- **Goal:** Repeatable sweep.
- **Focus change:** Produce a consistent case folder structure.
- **Tasks (3–25 min):**
 - Generate a list of 10 parameter sets.
 - Run each case to completion (or record failure).
 - Save `params.json` or a text file in each case folder.
- **Deliverable:** Folder tree with ODBs + logs + params.

Day 19: Full Pipeline Run (Build → Run → Post)

- **Goal:** One-command workflow.
- **Focus change:** Create a pipeline script that calls all stages.
- **Tasks (3–25 min):**
 - Implement `04_pipeline.py` to build/run/post in order.
 - Ensure outputs land in the correct per-case directories.
- **Deliverable:** Running pipeline produces `summary.csv`.

Day 20: Refactor + Checklist + README

- **Goal:** Consolidate fundamentals into a reusable template.
- **Focus change:** Remove duplication and document conventions.
- **Tasks (3–25 min):**
 - Refactor repeated code into `utils.py`.
 - Standardize naming (models, parts, sets, steps, jobs).
 - Write a short `README.md`: how to run, inputs, outputs, troubleshooting.
- **Deliverable:** Clean starter framework ready for your real models.